Kotlin Programming Language Evaluation Report

I. Suitability for the Problem

Kotlin is suitable for implementing the core functionality of the application because it supports numerical computation, conditional logic, functions, classes, and built-in collections. In the implementation, the exchange rates are modeled using a data class that is held in an immutable list, while the account state is held in a small class. These features allow the application to manage the account balance, perform currency conversions through the base currency (PHP), and compute projected daily balances with interest without requiring any external libraries. Kotlin is also a general-purpose language, so the same code base can grow into a larger banking system if needed. However, the current implementation keeps its data in memory only, so adding persistent storage or multiple accounts would require extra work, such as a serialization library or a database.

Code Block 1. Currency data class and the list of supported currencies

```
data class Currency(val name: String, val code: String, var rate: Double)

val currencies = listOf(
    Currency("Philippine Peso (PHP)", "PHP", 1.00),
    Currency("United States Dollar (USD)", "USD", 62.00),
    Currency("Japanese Yen (JPY)", "JPY", 0.40),
    Currency("British Pound Sterling (GBP)", "GBP", 84.00),
    Currency("Euro (EUR)", "EUR", 72.00),
    Currency("Chinese Yuan Renminbi (CNY)", "CNY", 9.00),
)
```

Each currency is a data class holding its name, code, and exchange rate (PHP per one unit). Only the rate is declared with var, since it can be changed through the Record Exchange Rates option. The compiler automatically generates equals(), toString(), and copy() for the data class, so no boilerplate is needed. PHP has a rate of 1.00 and acts as the base currency.

Code Block 2. Account class

```
class Account {
    var name: String? = null
    var balance: Double = STARTING_BALANCE
    val currency: Currency = baseCurrency
}
```

The account stores the registered name, the balance, and the account currency. The balance is always kept in PHP, so deposits and withdrawals made in another currency are first converted. The name is a nullable String? that stays null until the user registers, which lets the program know whether registration has happened yet.

Code Block 3. Conversion and rounding helpers

```
fun Double.money(): String = "%.2f".format(Locale.US, this)

fun Double.roundMoney(): Double =
    BigDecimal.valueOf(this).setScale(2, RoundingMode.HALF_UP).toDouble()

fun convert(amount: Double, from: Currency, to: Currency): Double =
    amount * from.rate / to.rate
```

Any currency can be converted to any other by passing through PHP: the amount is multiplied by the source rate and divided by the target rate. This makes it possible to convert between two foreign currencies, such as USD to JPY, using the same function.

II. Expressiveness and Readability

Kotlin offers concise and expressive syntax. Type inference, extension functions, string templates, default and named arguments, trailing commas, and single-expression functions remove much of the ceremony found in older languages. Programmers coming from Java, C#, or Swift will find the syntax familiar. Conciseness can still hurt readability when it is overused, such as with deeply chained expressions or heavy use of scope functions, so the code in this project favors short, named functions over clever one-liners.

Code Block 4. How valid menu selections are checked

```
fun promptSelection(label: String, range: IntRange): Int {
    while (true) {
        val value = prompt(label).removeSurrounding("[", "]").toIntOrNull()
        if (value != null && value in range) return value
        println("Invalid option. Enter a number from ${range.first} to ${range.last}.")
    }
}
```

This single function validates every numbered menu in the program. It accepts both "2" and "[2]", converts safely with toIntOrNull(), and uses the in operator with an IntRange to check that the number is a valid option. String templates (${...}) embed values directly in the message.

Code Block 5. Extension functions for formatting and rounding

```
println("Updated Balance: ${account.balance.money()}")
```

Because money() and roundMoney() are declared as extension functions on Double (see Code Block 3), they read like built-in methods at the call site. The code reads from left to right and hides the formatting details.

Code Block 6. Transactions as classes behind a common interface

```
interface Transaction {
    val returnsToMenuAutomatically: Boolean get() = false
    fun execute()
}

val transactions: Map<Int, Transaction> = mapOf(
    1 to RegisterAccount(account),
    2 to Deposit(account),
    3 to Withdraw(account),
    4 to CurrencyExchange(),
    5 to RecordExchangeRate(),
    6 to ShowInterestAmount(account),
)
```

Each menu option is a class that implements Transaction, and a Map links menu numbers to those objects. The main loop therefore does not need a long when or if-else chain; it simply calls transactions.getValue(choice).execute(). The interface also gives a default value for returnsToMenuAutomatically, which CurrencyExchange overrides.

Code Block 7. For-loop that builds the interest table

```
var balance = account.balance
for (day in 1..days) {
    val dailyRate = ANNUAL_INTEREST_RATE / DAYS_PER_YEAR
    val interest = (balance * dailyRate).roundMoney()
    balance = (balance + interest).roundMoney()
    println("%-3d | %-8.2f | %.2f |".format(Locale.US, day, interest, balance))
}
```

The range 1..days makes the loop easy to understand, and the body follows the formula in the specification: Daily Interest = End-of-Day Balance x (Annual Interest Rate / 365). The account balance is not modified, since the table only shows a projection.

III. Reliability and Safety

Kotlin offers several mechanisms for building reliable software. It is statically typed, so type mismatches are caught at compile time. Its type system also separates nullable types (String?) from non-nullable types (String), which prevents most null pointer errors. Val declarations, immutable collections, and exhaustive when expressions further reduce accidental state changes. The JVM provides automatic memory management through garbage collection, and exceptions are available through try/catch. Kotlin also comes with the kotlin.test library and works with JUnit for unit testing.

These features do not make a program correct by themselves. The programmer must still reject invalid input and enforce banking rules, such as negative or zero amounts, non-numeric text, and withdrawals larger than the available balance. In this project, validation uses the toIntOrNull() and toDoubleOrNull() functions instead of try/catch, which keeps the control flow simple and avoids exceptions for ordinary user mistakes.

Code Block 8. Null safety and safe parsing of input

```
fun prompt(label: String): String {
    print(label)
    return readlnOrNull()?.trim() ?: exitProcess(0)
}

fun promptAmount(label: String): Double {
    while (true) {
        val value = prompt(label).replace(",", "").toDoubleOrNull()
        if (value != null && value.isFinite() && value > 0) return value
        println("Invalid amount. Enter a number greater than 0.")
    }
}
```

readlnOrNull() returns null when the input stream ends, and the Elvis operator (?:) handles this by exiting cleanly instead of throwing. promptAmount() rejects text, NaN, infinity, zero, and negative numbers before any balance is touched. Using readln() with toInt() or toDouble(), as in the earlier Basic IO version, would crash the program on bad input.

Code Block 9. Guarding transactions against invalid states

```
fun Account.requireRegistered(): Boolean {
    if (name == null) {
        println("No account registered yet. Select [1] Register Account Name first.")
        return false
    }
    return true
}

if (debited > account.balance) {
    println("Insufficient balance. Withdrawal cancelled.")
    return
}
```

Deposit, withdrawal, and interest operations check that an account exists, and a withdrawal is cancelled if it exceeds the balance. Amounts are rounded with BigDecimal and RoundingMode.HALF_UP to avoid the stray fractions of a centavo that floating-point arithmetic can produce. The balance is still stored as a Double, which is acceptable for a course project but would be replaced by BigDecimal or integer centavos in a real banking system. The current program also supports only one account, so duplicate account names cannot occur, but this would have to be checked if multiple accounts were added.

IV. Tooling and Ecosystem

Kotlin is developed by JetBrains, and IntelliJ IDEA offers first-class support for it, including code completion, inline error highlighting, refactoring, a built-in debugger, and a formatter. Kotlin code can also be compiled from the command line using kotlinc, for example kotlinc MCO1_BasicIO_Kotlin.kt -include-runtime -d app.jar followed by java -jar app.jar. Since Kotlin compiles to JVM bytecode, a JDK must be installed. The

compiler's error messages are clear and often suggest a fix, and the debugger lets the developer set breakpoints and inspect variables, which makes tracing the interest loop or a failed conversion straightforward. The online Kotlin Playground is useful for testing small snippets.

The ecosystem is large because Kotlin is fully interoperable with Java libraries. Gradle and Maven manage dependencies, and the standard library already covers collections, string handling, and input, so this project needed no external packages. If the project were extended, kotlinx.serialization could save accounts to JSON, Ktor or Spring could expose the banking logic as a service, and tools such as ktlint and detekt could enforce style. The official documentation at kotlinlang.org is thorough and includes runnable examples.

V. Performance and Scalability

Kotlin programs are compiled to JVM bytecode and benefit from the JVM's just-in-time compiler, so performance is close to Java. For this application, every requirement runs instantly: currency conversion is a single multiplication and division, deposits and withdrawals are constant-time operations, and the interest table is a loop that grows linearly with the number of days. The amount of memory used is very small because there is one account and six currencies. The main cost is the JVM start-up time, which is noticeable for a short command-line program but does not matter for the work being done.

For scalability, Kotlin has coroutines for lightweight concurrency and can use any JVM concurrency tools, so it can handle many simultaneous users if the project grows. However, the present design would need changes first. The state is held in a single in-memory Account object, the exchange rates are mutable globals, and nothing is thread-safe or persisted. Supporting concurrent users would require synchronization or a database, and care would be needed to keep balances consistent.

VI. Comparison and Reflection

Compared with Java, which is also allowed by the project specification, Kotlin offers the same runtime, libraries, and tooling with considerably less code. The Currency data class replaces what would be a class with a constructor, getters, setters, equals(), hashCode(), and toString() in Java. Extension functions, string templates, default values in interfaces, and the in operator also make the source shorter. Both languages are statically typed and run on the JVM, so errors are caught at compile time, but Kotlin's null safety moves a whole category of runtime errors into the compiler, whereas Java's NullPointerException is only discovered at runtime. Java, in turn, has a longer history, more learning material, and is more widely taught, and its verbose style can be easier to follow for programmers who are not familiar with Kotlin's shorthand.

Kotlin provides enough functionality for the required banking operations, currency exchange, and interest computation. Challenges included the need for a JDK and compiler setup for command-line use, the JVM start-up delay, and handling floating-point rounding for money. Possible improvements are using BigDecimal for all balances, saving accounts to a file, supporting several accounts, and adding automated tests with kotlin.test for the conversion and interest formulas. Overall, Kotlin is a good choice for this project and for a larger version of it, and the main trade-off is learning its idioms compared with the more familiar Java syntax.